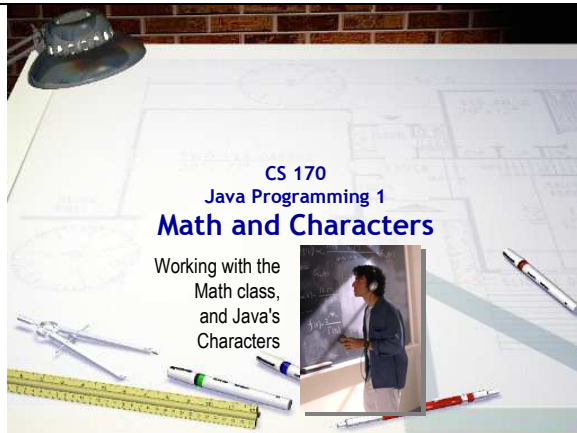




Slide 1
CS 170
Java Programming 1
Math and Characters
Duration: 00:00:45
Advance mode: Auto

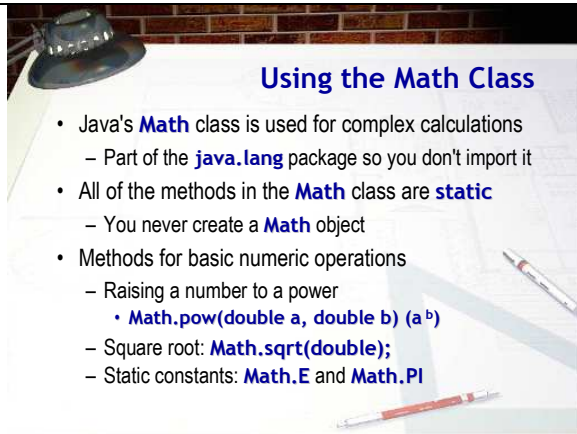
Notes:

In earlier lessons you looked at Java's arithmetic operators, and learned how to write expressions. Many calculations, however, require more than a simple expression. To meet this need, most programming languages have libraries, collections of reusable functions that are used to build more complex software.

In this lesson you'll look at the `java.lang.Math` class which contains methods for numeric operations such as calculating roots, sines and exponents.

You'll also be introduced to the Java primitive type designed to hold characters, the `char` type.

Let's start with the `Math` class.



Slide 2
Using the Math Class
Duration: 00:02:03
Advance mode: Auto

Notes:

Java's `Math` class contains a set of functions designed for calculations too complex for the arithmetic operators alone. to be .The `Math` class is part of the `java.lang` package. This package is automatically loaded and referenced by your Java Virtual Machine. You don't have to add any import statements to use any of the classes contained in `java.lang`.

All of the methods in the class are known as class or static methods. With a static method, instead of sending messages to objects, the receiver is the class itself. To use a method in the `Math` class, you don't need to create a `Math` object to; instead, you just use the name of the class when you call the method. (Actually, you can't create a `Math` object, because the class has no constructor.)

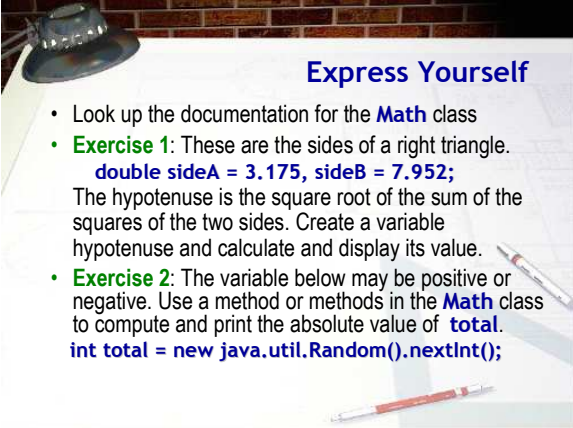
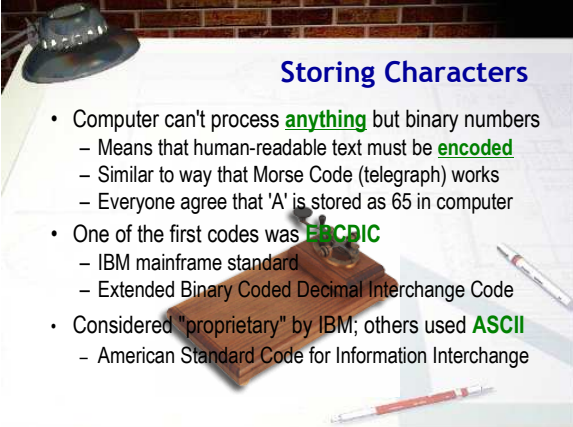
The `Math` class contains the normal set of functions and constants you'd expect in a rudimentary math library. Java doesn't have a exponential operator, for instance, but it does have a method named `pow()` that will raise any number to a power. Here's its signature:

```
Math.pow(double a, double b)
```

The method returns a raised to the `b`th power.

Another method you may commonly use is the `Math.sqrt()` method, which returns the square root of its argument. You may also want to make use of the static final (class) constants: `Math.E` (which contains the base of the natural logarithms), and `Math.PI`.

There are also several overloaded versions of `min()` and `max()`, for finding the minimum and maximum of two values, `abs()`, for finding the absolute value of a number, and `round()`, for converting a floating-point number to its closest integer value.

	Less frequently used methods (depending upon your interests, of course) include the methods for performing logarithm and trigonometric functions. You'll find a list in Table 2 on page 150 in your text and more detail in Appendix C, page 1077. For the definitive detail, though, you should look up the JavaDocs. Let's do that now.
 Slide 3 🎧 Express Yourself Duration: 00:00:60 Advance mode: Auto	Notes: Go ahead and create a new section in your IC lab document. Then, open BlueJ and look up the documentation for the Math class. The two variables below represent the sides of a right triangle. Type them into Code Pad. The hypotenuse is the square root of the sum of the squares of the two sides. Create a variable hypotenuse and calculate and display its value. Use the Math.pow() function to calculate the squares, even though simple multiplication would actually be more efficient. Shoot me a screen shot. For Exercise 2, I'd like you to use a method in the Math class to calculate and print the absolute value of a random number. Type the line here that creates the variable total into Code Pad. Then, on the next line, calculate and print its absolute value. Shoot me another screen shot.
 Slide 4 🎧 Storing Characters Duration: 00:01:11 Advance mode: Auto	Notes: On the first night of class, when we worked with the Debug program, you saw that everything stored in the computer: numbers, characters, even the colors for printing on the screen, is stored as binary numbers. To store character data, like the text file you created that night, your computer uses an agreed-upon correspondence or mapping between a number and a character, similar to the encoding used in Morse Code. There's nothing intrinsic about this mapping: as long as everyone agrees that the capital A is stored as 65, that's all that matters. Rather than use Morse-Code, invented for the telegraph, computer makers invented new codes. One of these was EBCDIC, invented by IBM for use on their mainframe computers. IBM considered EBCDIC a proprietary technology, so the American Standards Institute (ANSI), created an "open source" alternative, called ASCII (The American Standard Code for Information Interchange), which has been, more or less, the standard in the PC and Unix worlds.

- ASCII was designed to be efficient
 - Most characters stored in 7-bits
 - Small size meant quick transmission
- Modifying it for non-English languages was difficult
 - The "extra bit" used to provide alternate characters
 - "Code-pages" allowed DOS programs to switch
- Better solution is to use more memory than a byte
 - Several different variations
 - Most of the world has standardized on Unicode
 - Go visit at www.unicode.org

Dec	Hex	Char	Dec	Hex	Char
32	20	Space	64	40	@
33	21	!	65	41	A
34	22	"	66	42	B
35	23	#	67	43	C
36	24	\$	68	44	D
37	25	%	69	45	E
38	26	&	70	46	F
39	27	'	71	47	G

Slide 5

Unicode

Duration: 00:02:30

Advance mode: Auto

Notes:

ASCII was designed to be efficient. Because it uses 7 bits to store its characters, the small size meant efficient transmission over the terminal lines in the mainframe world it was developed in. In ASCII for instance, the number 65 stands for the capital A. Here is a chart showing the standard ASCII characters and their numeric representations.

Because ASCII itself only uses 7 bits, that means it could only hold 127 characters, which made it difficult to modify for non-English languages, even when they used the Western Roman alphabet. There were a couple of solutions to this.

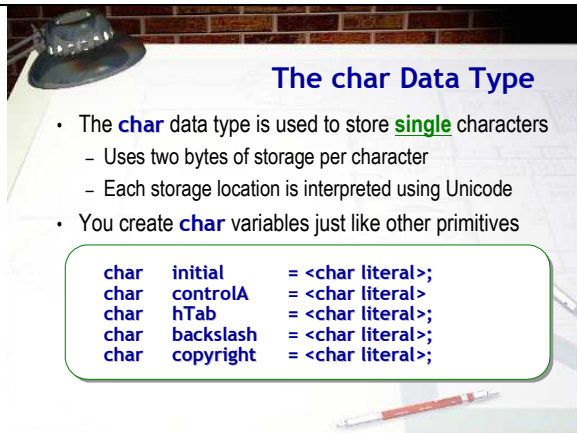
First, because each ASCII character can be stored in a single byte, there was 1 bit left over for some additional information. This additional bit has been put to a variety of uses. The early CPM/PC word processor, WordStar, used the extra bit to embed "soft" line breaks. With the introduction of the IBM PC, the extra bit was used to provide an additional 128 characters, including some widely used line-drawing characters.

MS-DOS used that extra bit to introduce the idea of Code Pages. With DOS you could switch the meaning of the "upper" 128 characters to support the accents needed for European languages as well as some non-European ones, such as Greek and Turkish. This was really a stop-gap solution, though.

Although ASCII met a real need for a quasi-universal character encoding during the first half-century of the computer age, today ASCII is really showing its age. The computer world is now much larger than it was in the past, especially with the Internet explosion. Not only are computer users using non-English languages, but their languages are often written using non-Roman alphabets; alphabets that won't fit into ASCII's seven or eight bits, no matter how much they are shoe-horned.

Several alternative character encoding schemes have been suggested to replace ASCII, and as a programmer you might have to deal with some of them, (such as DBC, and MBCS). By now, however, it looks like the winner in the race to succeed ASCII is the Unicode character set, which is, as you might suspect, is used by Java to represent characters. You can find more about Unicode at:

<http://www.unicode.org>



The char Data Type

- The **char** data type is used to store **single** characters
 - Uses two bytes of storage per character
 - Each storage location is interpreted using Unicode
- You create **char** variables just like other primitives

```
char initial    = <char literal>;
char controlA  = <char literal>;
char hTab      = <char literal>;
char backslash = <char literal>;
char copyright = <char literal>;
```

Slide 6 

The char Data Type

Duration: 00:01:02
Advance mode: Auto

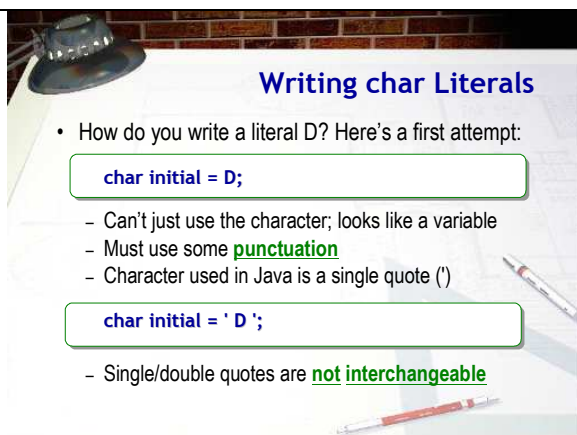
Notes:

Java stores characters using the char primitive data type. (Since char stands for character, you'll hear heated arguments about whether the name should be pronounced care, car, or, and even shar.)

Unlike the integer and floating-point types, the char family has only one member. Each char variable, (or literal), uses two bytes of storage. Each character is stored as an unsigned binary integer, and is interpreted by using the Unicode character set.

By now, the pattern you use to create a char variable should look familiar; it's the same one you used to create int and double variables. You create char variables like these, which represent my middle initial, two control characters: control A and the horizontal tab, the backslash and the copyright symbol.

Each of the variables shown here requires a slightly different form of initialization, even when we use character literals.



Writing char Literals

- How do you write a literal D? Here's a first attempt:

```
char initial = D;
```

- Can't just use the character; looks like a variable
- Must use some **punctuation**
- Character used in Java is a single quote (')

```
char initial = 'D';
```

- Single/double quotes are **not interchangeable**

Slide 7 

Writing char Literals

Duration: 00:01:22
Advance mode: Auto

Notes:

Let's start with the first, and easiest of these examples. How do you write a literal D, which is my middle initial? If you try simply inserting the character--which is what you did with literal integers and floating point numbers--you end up with this:

At first, this looks OK, but it turns out to have a fatal flaw. How does the compiler know whether you want the character literal D, or the variable named D? Since D is a valid name for a variable, the compiler goes searching for it, comes up empty, and issues a stern rebuke.

The solution is easy. To placate the Java compiler, we simply have to add some punctuation to our character to let the compiler know that it is, in fact, a character literal. The punctuation you use is the single quote (') character. With this new-found knowledge, it's obvious that our definition should be written as: `char middleInitial = 'D';`

When you write a character literal, you can only enclose one single character at a time; also, make sure you use the single quotes, not the double quotes, even if you have only a single character. If you use double-quotes, you'll create a String literal, not a char literal. Strings are objects, not primitive types, and so Java won't permit this assignment.

Other char Literals

- Some characters are not "printable"
 - ASCII 0-31 traditionally used to control hardware
 - Thus, named "control characters"
- How do you include such characters in your code?
 - Generated by holding down CTRL+ Character
 - Still used to "control" software
 - Can write these by simply using the integer value

```
char ctrlA = 1;
```

Slide 8 🎧
Other char Literals
 Duration: 00:01:36
 Advance mode: Auto

Notes:

Writing a literal D proved to be quite simple, but how about the other characters? Let's start with controlA.

The first 128 characters in the Unicode character set are the same as those used by ASCII. (You can find the first 256 Unicode characters, called the Latin-1 Subset in Appendix B in your text.) In ASCII, (and in Unicode), the first 32 characters are called control characters. These are non-printing characters that were given the name control characters because they were used to control terminals and printers during the early days of computing.

To produce a control character, you hold down the Ctrl key on your keyboard (much like you would with the Shift key), and press one of the alphabetic characters A-Z. This yields the control characters whose ASCII/Unicode values are 1-26.

Control characters still are used to control the programs you run. If you open your word processor and start typing, when you type Ctrl+S, instead of inserting the character with the Unicode value 19 (Ctrl-S), your word processor is likely to save your file instead.

As you may have guessed, you can initialize controlA by just using its integer Unicode value like this: `char controlA = 1;` // Unicode 1 is Control-A

A better way, (because it makes your intentions clearer), is to use a Unicode escape sequence, which you'll meet in the next few sections.

"Special" Characters

- Java has a second way to enter "special" characters
 - Called an **escape sequence**
 - These include some control characters and punctuation
- An escape sequence has two parts
 - The **escape character**
 - In Java, the escape character is the backslash (\)
 - Tells the compiler "Look out - special instructions ahead!"
 - Followed by a **translation value**

Slide 9 🎧
"Special" Characters
 Duration: 00:01:36
 Advance mode: Auto

Notes:

The next character we need to initialize is called hTab which stands for horizontal tab. If you look back at the ASCII chart, or at the first page of the Unicode character charts, you'll see that many of the control characters have names, which allude to their functions under traditional computer systems. For instance, the character Unicode 7 (Ctrl-G) is also called the BEL character, because it produced audible output on the early teletypes.

Using the same scheme, Unicode character 8 was the BS (back-space) character, and Unicode character 9 was the horizontal tab character, or HTAB, the character produced by pressing the Tab key on your keyboard.

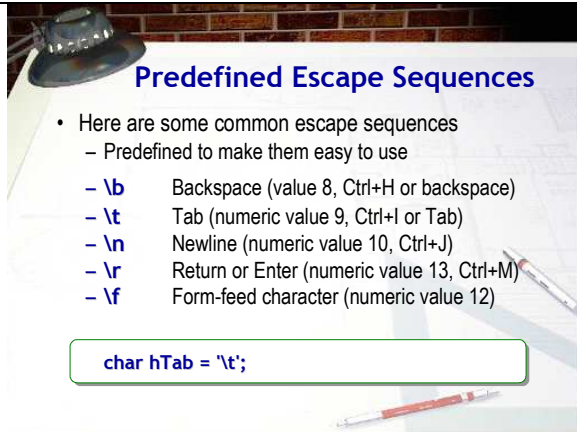
Rather than remembering which code belonged to which key, however, programmers came up with a shorthand for these keys called an escape sequence. An escape sequence is a method of writing "difficult" characters, that is, characters that are invisible or that are used, normally to delimit other characters.

The first part of an escape sequence is to designate one character as the "escape character". In Java, the escape character is the backslash. When that character is

encountered--in a character literal or in a group of characters, called a String--the compiler treats it as a flag that means:

"Look out! The next character is special."

Whenever a backslash appears in a character constant, the following character is set aside for special treatment and is translated in a special way.



Predefined Escape Sequences

- Here are some common escape sequences
 - Predefined to make them easy to use
 - **\b** Backspace (value 8, Ctrl+H or backspace)
 - **\t** Tab (numeric value 9, Ctrl+I or Tab)
 - **\n** Newline (numeric value 10, Ctrl+J)
 - **\r** Return or Enter (numeric value 13, Ctrl+M)
 - **\f** Form-feed character (numeric value 12)

```
char hTab = '\t';
```

Slide 10

Predefined Escape Sequences

Duration: 00:01:40

Advance mode: Auto

Notes:

Java maintains "special predefined notation" for several of the common control characters, as escape sequences. These include:

- **\b** : The backspace character (Which has the Unicode value 8, but can also be generated by pressing Control + H or the Backspace key on your keyboard).
- **\t** : The horizontal tab character (Unicode value 9 but generated by pressing either the Tab key or Control + I)
- **\n** : The newline character (Unicode value 10 or Control + J)
- **\r** : The carriage-return (ENTER) (Unicode 13 or Control + M. Note that on Windows, pressing the Enter key actually generates two characters: the carriage return, followed by the newline. On Unix and the Mac only a single character is generated. Unix used the newline and the Mac uses the carriage return.)
- **\f** : The form-feed character (Unicode 12)

With this information, it's easy to see how to initialize the hTab variable, using one of the special escape characters:

```
char hTab = '\t';
```

Notice that when you use an escape sequence to initialize a char variable, that you use single quotes around the whole sequence. Even though the escape sequence contains two characters (the backslash and the t), the compiler turns the compiled code into a single character.

Unicode Escape Sequences

- Escape sequences represent any Unicode character
- Start with `\u` or `\U`, then 4-digit hex Unicode value
- Use escapes to create `char` variables or in strings

```
char controlA = '\u0001';
char tradeMark = '\u2122';
char copyright = '\u00A9';
String message = "\u00BC is \u00BD of \u00BD\n";
```

- Difference between characters and glyphs

Slide 11

Unicode Escape Sequences

Duration: 00:02:01

Advance mode: Auto

Notes:

In addition to using one of the special predefined escape values, you can use a Unicode escape sequence to represent any character in the Unicode character set. A Unicode escape sequence starts with a backslash-u and is followed by the four-digit, hexadecimal Unicode value, which you can look up at the Unicode Web site.

Once you have the completed value, you put the whole thing in single quotes, just like a regular character literal. Here are some examples:

```
char controlA = '\u0001';
char tradeMark = '\u2122';
char copyright = '\u00A9';
```

Note that you can also embed Unicode escape characters inside a String. When you do that, you don't put the single quotes around the escape sequence; you just wrap the whole String inside double quotes.

Java allows you to use any of the Unicode characters in your programs, by simply following the examples above. However, you may be disappointed to find that not all of the characters show up in your programs. That's because even though Java stores the characters internally using Unicode, when it displays the characters, it uses the glyphs of your operating system's native font. If the native font is Unicode aware, (that is, if it has "pictures" or glyphs for all of the Unicode characters), your program will display correctly. If, however, it only contains the basic ASCII characters, then the others will not display.

In the example shown here, for instance, each of these characters has exactly the same Unicode value, (97 for the lowercase 'a'), but the glyphs are completely different for each one. This is further complicated when it comes to Java's widget sets. Even if you use a Unicode font in your program, your platform's widgets, (like the AWT Label and Button classes), may not support Unicode. The more modern widgets, like the JFC JLabel and JButton usually do support Unicode, however.

Escaping Punctuation

- Store an apostrophe inside a `char`

```
- char apostrophe = '\';
```
- Store a quote inside a `String`

```
- String msg = "and then he said, \"Hello!\"";
```
- Store a backslash inside a `String`

```
- String iniFile = "C:\\Windows\\my.ini";
```

Notes:

Finally, escape sequences can be used to embed characters that Java usually uses for its own purpose such as the apostrophe, the double quote and the backslash itself.

To store an apostrophe inside a char, you'll need to use three apostrophes, with the second one preceded by the escape character like this. If you want to put an apostrophe inside a String you don't have to escape it (but you can, of course).

To store a double-quote inside a String, simply precede it with the escape sequence like this. Sometimes this is confusing. For instance, this string ends with two double-quotes. The first one is escaped, so it is embedded inside the String; the last

Slide 12

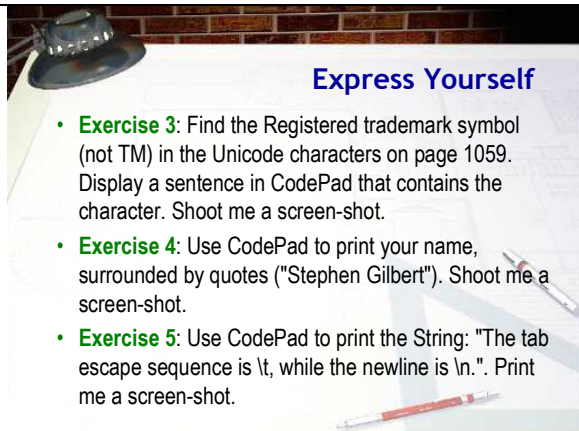
Escaping Punctuation

Duration: 00:01:30

Advance mode: Auto

one simply ends the String itself. To embed a double-quote inside a char variable, you don't need to escape it, just surround it with single quotes.

Finally, you might find it necessary sometimes to embed a backslash itself inside a String or a character, especially since Windows decided to use the backslash as its path separator. For that, you need to use two backslashes as in the example here. If you're a network guru, make sure you don't confuse this with the Windows UNC path names, like those used on a LAN and which also start with a double backslash. If you have a String containing a UNC path name, you'll need four backslashes to produce the two backslashes at the beginning of the path.



Express Yourself

- **Exercise 3:** Find the Registered trademark symbol (not TM) in the Unicode characters on page 1059. Display a sentence in CodePad that contains the character. Shoot me a screen-shot.
- **Exercise 4:** Use CodePad to print your name, surrounded by quotes ("Stephen Gilbert"). Shoot me a screen-shot.
- **Exercise 5:** Use CodePad to print the String: "The tab escape sequence is `\t`, while the newline is `\n`". Print me a screen-shot.

Slide 13

Express Yourself

Duration: 00:00:54

Advance mode: Auto

Notes:

OK, now it's your turn. Use CodePad to complete these exercises, shooting me a screen-shot for each one.

For Exercise 3, look in the Unicode character on page 1059 in your text (or on the Unicode Web site), and find the Registered trademark symbol (not the TM symbol). Then, display a sentence in Code Pad that contains the symbol.

For Exercise 4, use Code Pad to print your name, surrounded by quotes like this. (Please don't print my name, though.)

Finally, for Exercise 5, use Code Pad to print the String: "The tab escape sequence is `\t`, while the newline is `\n`."